



CHAT TIME: A MOBILE DEVELOPME NT CASE STUDY.

Plumbz

PRESENTED BY: PLUMBY

Objective

ChatTime is a chat app for mobile devices that built with React Native. The app will provide users with a chat interface and option to save images and their location.

```
import { Bubble, GiftedChat, InputToolbar } from "react-native-gifted-chat";
import { useState, useEffect } from 'react';
import { StyleSheet, View, Platform, KeyboardAvoidingView } from 'react-native';
import { collection, addDoc, onSnapshot, query, orderBy } from "firebase/firestore";
import AsyncStorage from "@react-native-async-storage/async-storage";
import MapView from 'react-native-maps';
import CustomActions from '../CustomActions';

const Chat = ({ db, route, navigation, isConnected, storage }) => {
  const { userID, name, backgroundColor } = route.params; //handle the props
  const [messages, setMessages] = useState([]);
  const onSend = (newMessages) => {
    addDoc(collection(db, "messages"), newMessages[0])
  }

  let unsubMessages;

  // Render the messages
  useEffect(() => {
    if (isConnected === true) {
      // unregister current onSnapshot() listener to avoid registering multiple listeners when
      // useEffect code is re-executed.
      if (unsubMessages) unsubMessages();
      unsubMessages = null;
      const q = query(collection(db, "messages"), orderBy("createdAt", "desc"));
      unsubMessages = onSnapshot(q, (documentsSnapshot) => {
        let newMessages = [];

        documentsSnapshot.forEach(doc => {
          newMessages.push({
            id: doc.id,
            ...doc.data(),
            createdAt: new Date(doc.data().createdAt.toMillis())
          })
        })
        cacheMessages(newMessages);
        setMessages(newMessages);
      })
    } else loadCachedMessages();

    // Cleanup the listener when the component unmounts
    return () => {
      if (unsubMessages) unsubMessages();
    };
  }, [isConnected]);
```

DURATION

Developing the client-side of the chat app took more time than setting the backend. I focused on understanding how React Native components, navigation, and asynchronous data handling work together to create a smooth user experience. Special attention was given to integrating Firebase services like Firestore, Authentication, and Storage, while also ensuring offline functionality and accessibility.

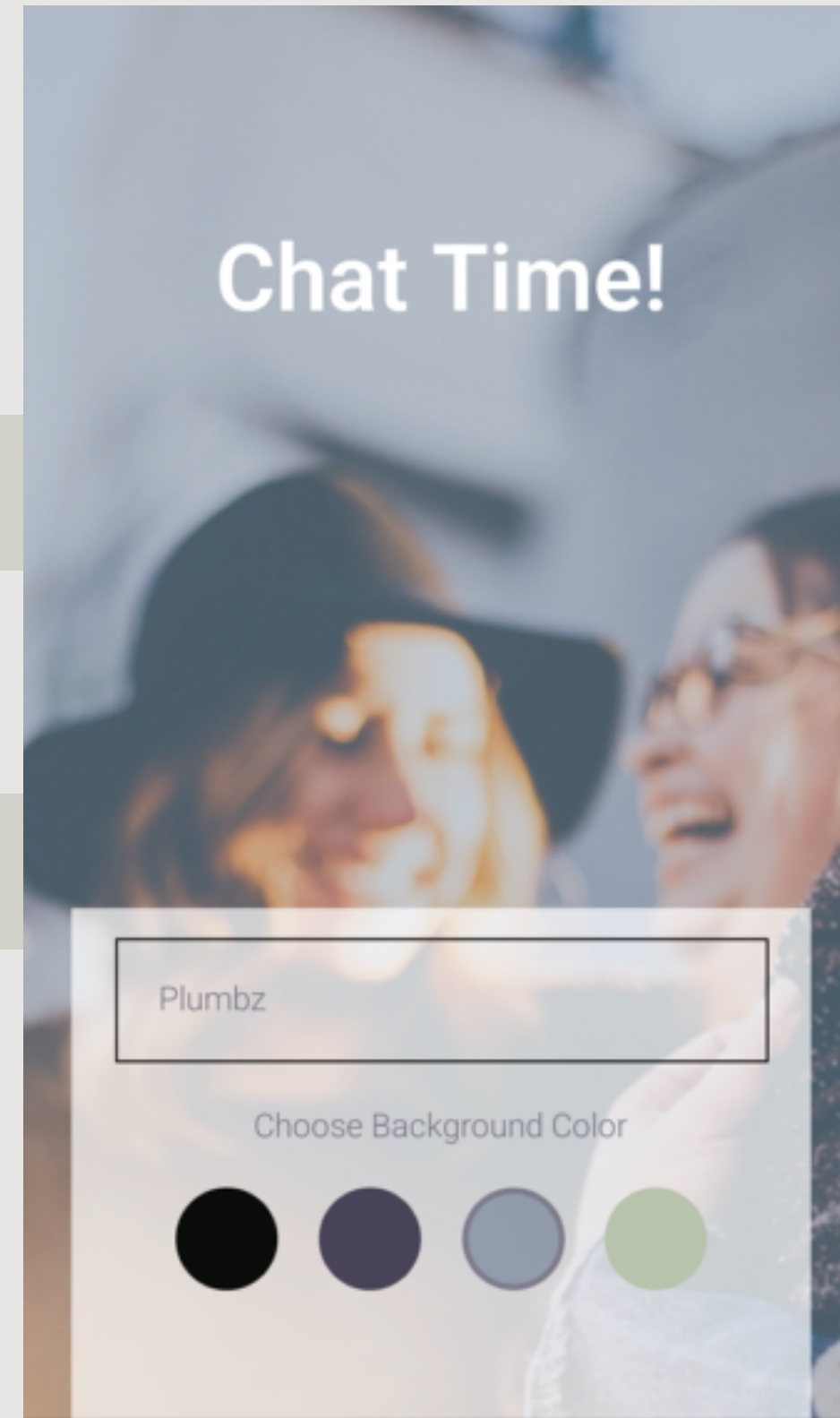
METHODOLOGIES & TOOLS

- REACT NATIVE
- EXPO
- GIFTED CHAT (CHAT UI LIBRARY)
- GOOGLE FIREBASE (FIRESTORE DATABASE, CLOUD STORAGE, AUTHENTICATION)
- ASYNCSTORAGE (LOCAL STORAGE)
- ANDROID STUDIO (EMULATE ANDROID DEVICE)
- REACT NAVIGATION
- ACCESSIBILITY TESTING
- GIT AND GITHUB

DESIGN AND PLANNING

the project began with wireframing the user flow, from a welcome screen to a fully functional chat interface. Key consideration included:

- User authentication
- Storage for messages and media
- Offline functionality



DEVELOPMENT PROCESS

the app was developed using React native for the Frontend and Firebase for real-time database and storage. key steps included:

- setting up user authentication with Firebase Auth
- implementing real-time messaging with Cloud Firestore
- Integrating Firebase Storage for image and media uploads
- adding offline support using >AsyncStorage


```

import { StyleSheet, View, Platform, KeyboardAvoidingView } from 'react-native';
import { collection, addDoc, onSnapshot, query, orderBy } from "firebase/firestore";
import AsyncStorage from "@react-native-async-storage/async-storage";
import MapView from 'react-native-maps';
import CustomActions from './CustomActions';

const Chat = ({ db, route, navigation, isConnected, storage }) => {
  const { userID, name, backgroundColor } = route.params; //handle the props
  const [messages, setMessages] = useState([]);
  const onSend = (newMessages) => {
    addDoc(collection(db, "messages"), newMessages[0])
  }

  let unsubMessages;

  // Render the messages
  useEffect(() => {
    if (isConnected === true) {
      // unregister current onSnapshot() listener to avoid registering multiple listeners when
      // useEffect code is re-executed.
      if (unsubMessages) unsubMessages();
      unsubMessages = null;
      const q = query(collection(db, "messages"), orderBy("createdAt", "desc"));
      unsubMessages = onSnapshot(q, (documentsSnapshot) => {
        let newMessages = [];

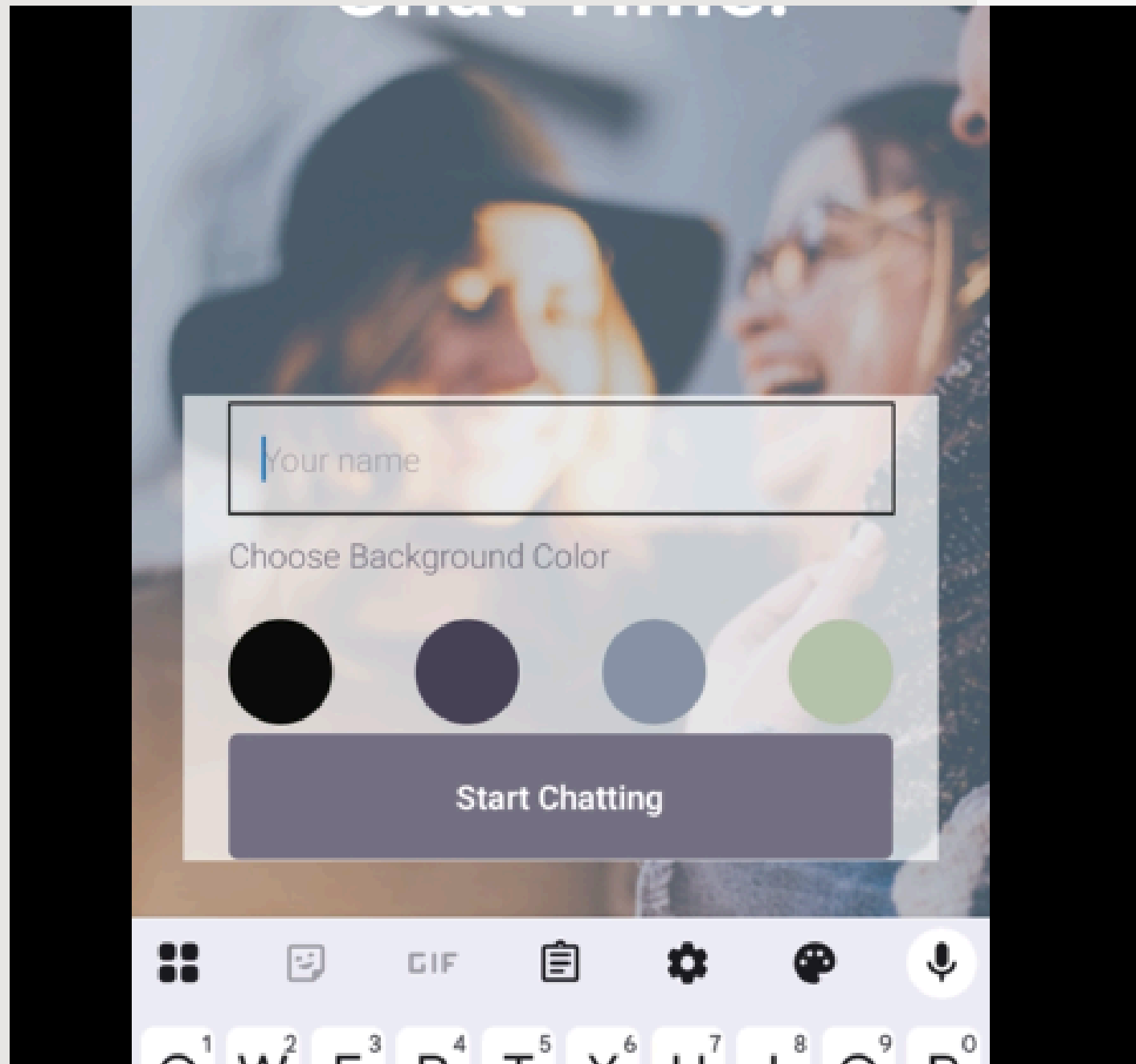
        documentsSnapshot.forEach(doc => {
          newMessages.push({
            id: doc.id,
            ...doc.data(),
            createdAt: new Date(doc.data().createdAt.toMillis())
          })
        })
        cacheMessages(newMessages);
        setMessages(newMessages);
      })
    } else loadCachedMessages();

    // Cleanup the listener when the component unmounts
    return () => {

```

SERVER-SIDE

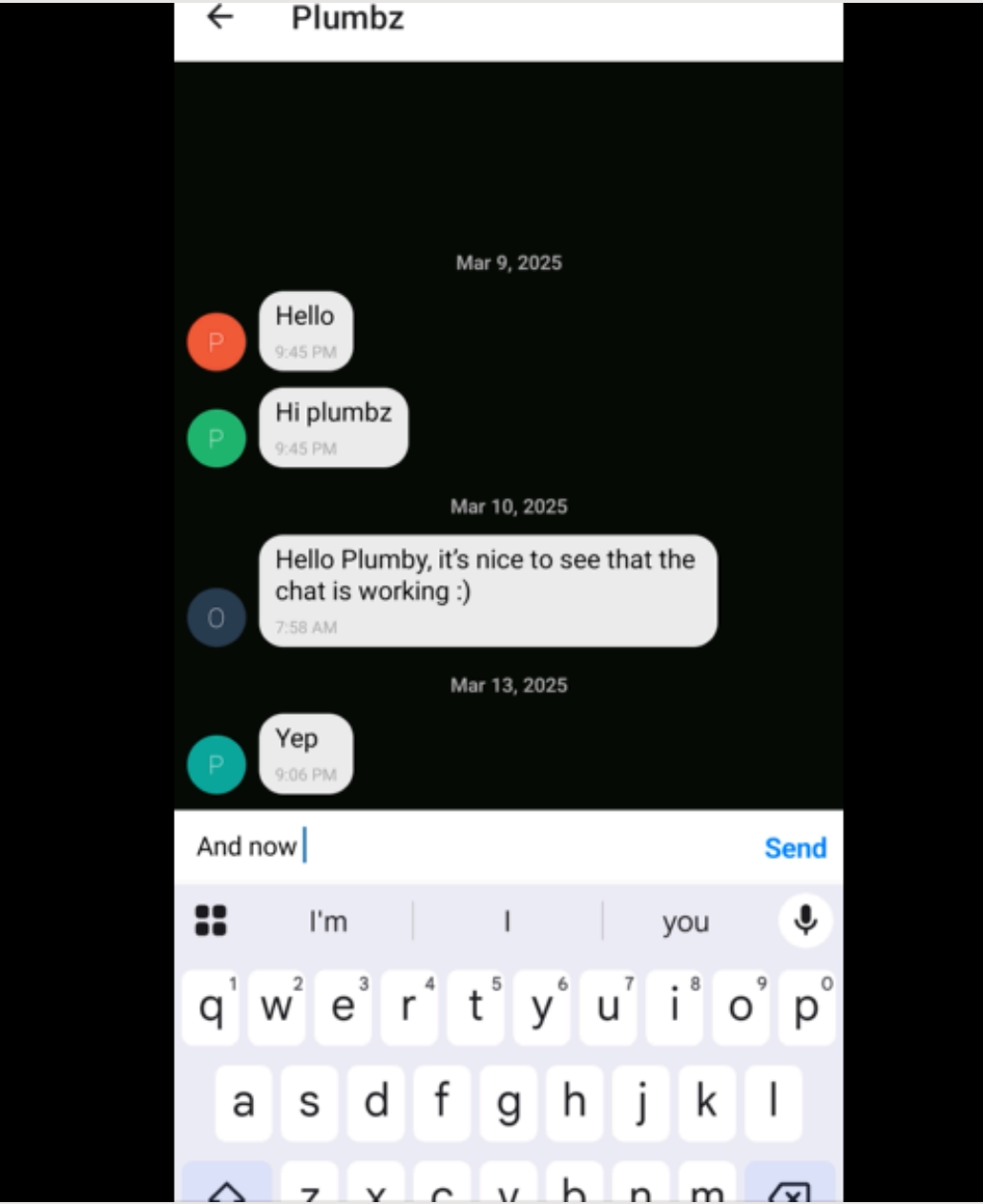
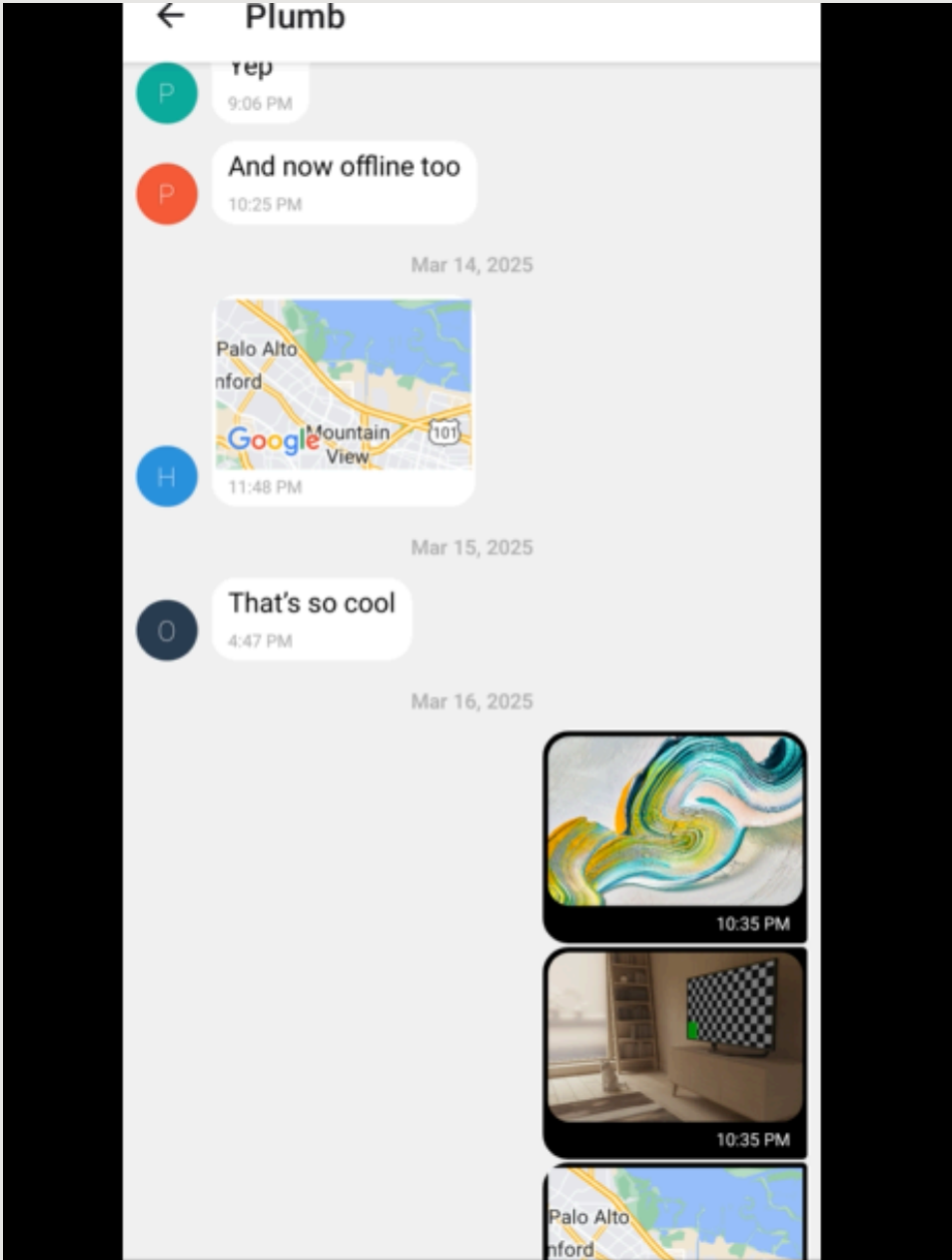
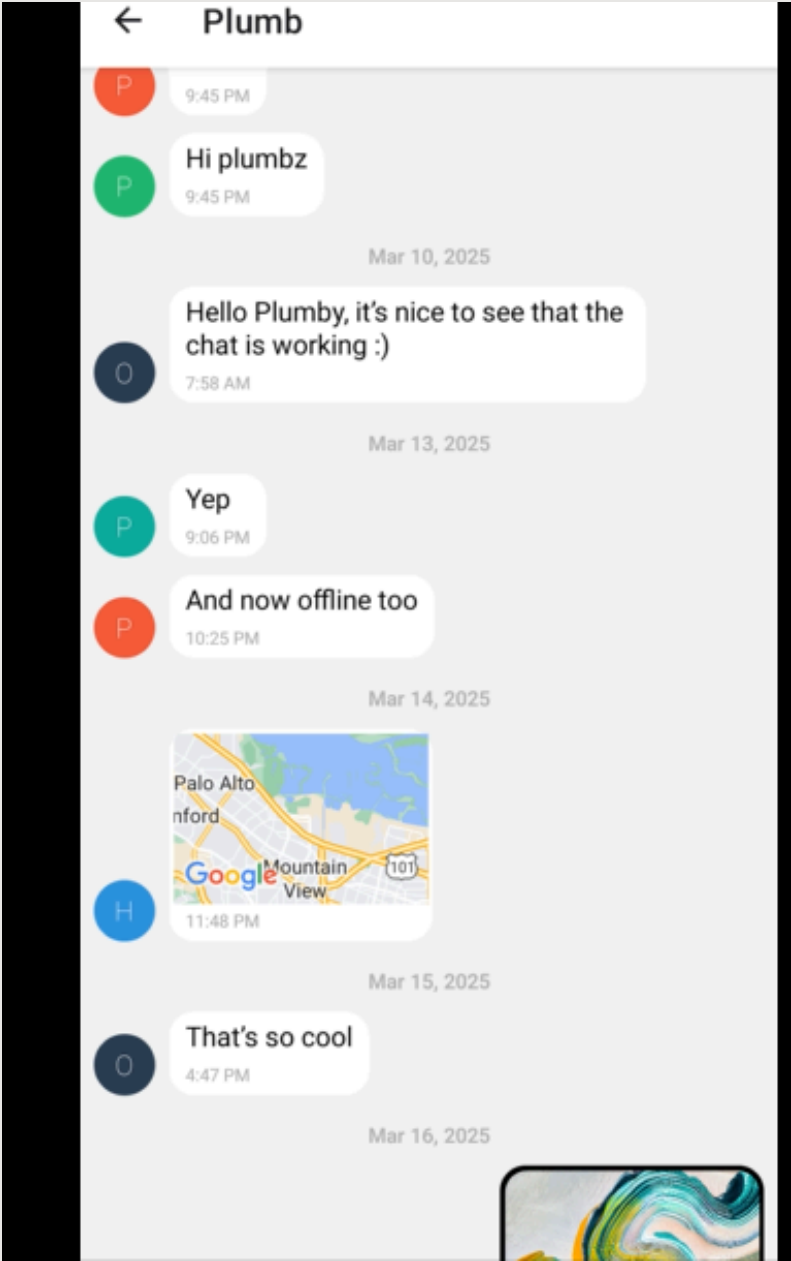
- Implemented Firebase BaaS to manage authentication, data storage, and media handling using built-in SDKs APIs
- Used Firestore (NoSQL) for real-time message syncing and Firebase Storage for secure image uploads
- ensure secure communication via HTTPS, storing and retrieving data in Json format



CLIENT SIDE

- Built with React native and Expo for cross-platform performance and a smooth user experience
- Integrated gifted chat, React navigation, and AsyncStorage to support messaging UI, screen transitions, and offline caching
- Connected to Firebase services for real-time messaging, image uploads, and location sharing via Google Maps API using JSON data format

PAGE-VIEWS



PAGE VIEW 2

